Seeing Your Code: An Interactive Visual Task Framework for LZSCC. 111 C Programmers

Andrei Florea 39603784
Supervisor: Dr Tobias Meggendorfer
LZSCC.300 Third Year Project, Lancaster University Leipzig

October 2026

Abstract

Students on LZSCC.111 Software Development learn to program in C, where program state is hidden and mistakes often show up only as a wrong number or a crash. This project will build a framework of interactive C exercises that increase in difficulty. In each exercise the student implements a single function. Provided scaffolding handles input and output, and a Python visualiser shows what the code does step by step, such as the bars of an array being searched or a robot moving through a maze. Student code runs inside a C harness that records each step as an event trace and catches crashes, infinite loops and out-of-bounds access. The framework will be evaluated technically and through a small study with students, comparing the exercises with and without the visualiser. The expected outcome is a reusable set of tasks for LZSCC.111, along with indicative evidence on whether visual feedback improves novice understanding and engagement.

1 Introduction

LZSCC.111 introduces students to programming in C. For a beginner, C gives little visible feedback. A loop that searches an array or a function that moves a robot produces at most a line of printed output, and the student cannot see how the program reached its result. Reading input, parsing it and formatting output also take effort unrelated to the concept being taught, so beginners spend their time on scanf and printf instead of the algorithm.

Research on algorithm visualisation has found that showing execution helps most when students interact with the visualisation rather than only watch it [1, 2]. This project applies that idea to LZSCC.111. Students write only the core logic of each task in C, and a Python visualiser shows its effect.

The rest of this proposal is organised as follows. Section 2 reviews related work and the gap this project addresses. Section 3 sets out the aims, objectives and research questions, followed by the methodology. Section 4 describes the programme of work, Section 5 lists the resources required, and the references follow.

2 Background

3 The Proposed Project

3.1 Aims and Objectives

The aim of the project is to build a reusable framework of visual C tasks of increasing difficulty, aligned with the LZSCC.111 curriculum, and to gather evidence on whether it helps students learn. The objectives are:

1. Review the literature on program visualisation, scaffolding and novice programming, and derive design principles for the framework.
2. Design the architecture that connects student C code to a Python visualiser through a defined event-trace format.
3. Build a C harness that runs student code safely, catching crashes, infinite loops and out-of-bounds errors and reporting them in plain language.
4. Implement at least four tasks of increasing difficulty, each with C scaffolding, a reference solution and a visualisation.
5. Package the framework with automated tests, continuous integration and documentation, so that teaching staff can add new tasks.
6. Evaluate the framework technically and through a small user study.
7. (Stretch goal) Make the tasks gradable in DOMjudge.

The project addresses two research questions:

RQ1 How can a visual feedback framework for C be designed so that students implement only the core logic, while new tasks remain easy to add?

RQ2 Do students who use the visual tasks report better understanding and engagement than when they use plain text-based exercises?

3.2 Methodology

Software engineering approach. The project will follow an iterative approach, with a working increment shown to the supervisor every two weeks. This suits the project because each task can be built, tried and adjusted on its own, and early feedback on the first task will shape the design of the rest. A waterfall approach was considered and rejected, since the requirements for later tasks depend on what is learned from the first prototype.

System design. The framework separates student code from visualisation through an event trace. The student writes one function. A C harness calls it, records each meaningful step, for example "compared index 3" or "robot moved north", and writes these events as JSON lines to standard output. The Python visualiser reads the trace and replays it as an animation. Two alternatives were considered: compiling student code as a shared library called through Python's `ctypes`, and a live socket connection between the processes. The first was rejected because a segmentation fault in student code would also terminate the visualiser. The second adds complexity without a clear benefit for beginners. A recorded trace also makes visualisations testable and lets students step backwards through execution.

Safe execution. First-year code often crashes, loops forever or reads outside array bounds. Student code will therefore run in a separate process with a time limit, compiled with sanitisers such as AddressSanitizer. When a fault occurs, the harness will stop the program and show a short explanation instead of a raw error.

Task design. The tasks will increase in difficulty and follow the order of topics in LZSCC.111. The planned tasks are listed in Table 1; the final order will be agreed with the supervisor once the weekly topic list is available. Increasing difficulty while gradually reducing scaffolding follows the engagement levels described by Naps et al. [2].

Task	What the visualiser shows	Concepts practised
1. Maximum of an array	Bars, with the current index and best value highlighted	Loops, arrays
2. Sorting	Each comparison and swap animated	Nested loops
3. Robot in a known maze	Robot following the student's move and turn commands	Functions, sequencing
4. Robot with sensors	Robot following walls using checks such as `wall_ahead()`	Conditionals, while loops
5. Robot pathfinding	Robot finding the shortest route, e.g. with breadth-first search	2D arrays, queues
6. Linked list (optional)	Nodes and pointers drawn as boxes and arrows	Structs, pointers

Table 1: Planned tasks.

Tools. Each task will be described by a specification file (JSON or YAML), so that adding a task requires only a specification, a C scaffold and a visualiser module. The visualiser will use either pygame or matplotlib, chosen after prototyping the first task in week 4. C code will be built with Make, and reference solutions and automated tests will run in continuous integration on every commit.

Participants and ethics. The user study will aim for 10 to 20 participants, ideally current LZSCC.111 students, otherwise students from the author's cohort. This size is realistic for the timeframe and enough for a descriptive comparison, though not for statistically conclusive results. Participants will receive an information sheet and sign a consent form, and all responses will be anonymised. Ethics approval will be requested early in Michaelmas term, so that the study can run in Lent term.

Pilot study. Before the main study, two or three students will complete the tasks and the questionnaire. The pilot should reveal any bugs in the tasks and unclear questions, so they can be fixed before the main study.

Evaluation. The evaluation has two parts, one for each research question. For RQ1, every task must accept its reference solution and catch a set of deliberately faulty solutions, such as an off-by-one error, an infinite loop and an out-of-bounds access. The time and lines of code needed to add a new task will also be measured. For RQ2, participants will solve tasks with and without the visualiser and then complete a questionnaire on understanding and engagement, including the System Usability Scale [6]. Where possible, a short concept quiz will be given before and after the session. Results will be analysed descriptively.

Limitations. The sample is small and drawn from a single course, so the user study can give only indicative evidence. Participants may also respond more positively because they know the author.

4 Programme of Work

The project runs from October 2026 to the end of Lent term 2027, in the following stages:

- **Research and proposal (weeks 1–3).** Literature review, first supervisor meetings, ethics application and this proposal.

- **Prototype (weeks 4–5).** Event-trace format, C harness and the first task working end to end, ready for checkpoint 1 in week 5.
- **Core tasks (weeks 6–10).** Tasks 2 to 4, automated tests and continuous integration, are ready for checkpoint 2 in week 10.
- **Pilot and user study (weeks 11–15).** Tasks 5 and 6, the pilot study, and then the main study, are in progress by checkpoint 3 in week 15.
- **Analysis and write-up (weeks 16–20).** Analysis of results, the final report and the presentation. Drafting of the report starts in week 6.

The schedule is shown in Figure 1. Weeks 1 to 10 are in Michaelmas term and weeks 11 to 20 in Lent term.

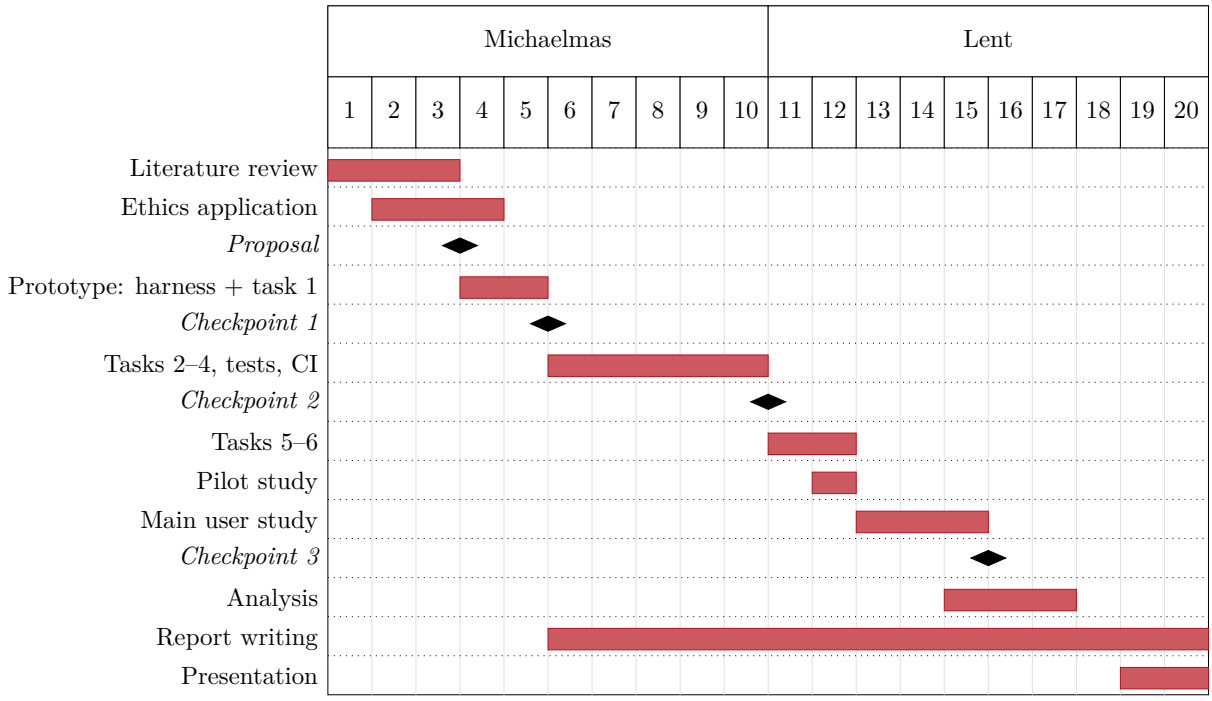


Figure 1: Project schedule by teaching week, 2026–2027.

5 Resources Required

- Access to the university lab machines, to test that the tasks build and run in the environment students use.
- A personal laptop with a C compiler, Python and Git for development.
- Volunteer participants from LZSCC.111 or the author's cohort for the pilot and main study, recruited with the supervisor's help.
- Ethics approval from the university for the user study.
- (Optional/Add-on) Access to a test instance of DOMjudge, to check task compatibility.

No paid software or special hardware is required.

References

[1] C. D. Hundhausen, S. A. Douglas and J. T. Stasko, "A meta-study of algorithm visualization effectiveness," *Journal of Visual Languages & Computing*, vol. 13, no. 3, pp. 259–290, 2002.

[2] T. L. Naps et al., "Exploring the role of visualization and engagement in computer science education," *ACM SIGCSE Bulletin*, vol. 35, no. 2, pp. 131–152, 2002.

[3] J. Sorva, V. Karavirta and L. Malmi, "A review of generic program visualization systems for introductory programming education," *ACM Transactions on Computing Education*, vol. 13, no. 4, pp. 1–64, 2013.

[4] P. J. Guo, "Online Python Tutor: embeddable web-based program visualization for CS education," in *Proc. 44th ACM Technical Symposium on Computer Science Education (SIGCSE)*, 2013, pp. 579–584.

[5] R. E. Pattis, *Karel the Robot: A Gentle Introduction to the Art of Programming*. New York: Wiley, 1981.

[6] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, P. W. Jordan et al., Eds. London: Taylor & Francis, 1996, pp. 189–194.

[7] DOMjudge, "DOMjudge documentation." Available: `https://www.domjudge.org/docs/`